

Kosmocrates HYPHAE

Metatron Microtopology Extension

Master Specification v0.4.1

Topological Surgery Laboratory for StructuralCrystals, NormGenes,
CorpusCartography and CollapsePlans

Author: Sebastian Klemm
29 May 2026

Document status. This document specifies the v0.4.1 Metatron Microtopology Extension for Kosmocrates HYPHAE. It preserves HYPHAE v0.4 doctrine: non-copying, evidence-first, replay-governed, Foundry-bound, Kosmocrates-governed and non-authoritative by existence. It integrates the existing Metatron-Scan graph kernel as a bounded microtopological analysis and planning kernel for HYPHAE regions.

Contents

Executive Summary	iv
Normative Language and Primary Terms	v
I Doctrine and Scope	1
1 Purpose and Scope	2
1.1 Purpose	2
1.2 Scope	2
1.3 Non-goals	2
2 Core Doctrine	3
3 Source Repository Neutralization	4
3.1 Inspected Repository Invariants	4
3.2 Interpretive Rebinding	4
II Metatron Microtopology Model	6
4 Architecture Position	7
5 TopologyRegionRef	8
6 Region Extraction	9
6.1 RegionExtractionProfile	9
6.2 MVP Extraction Profiles	9
7 ProjectionProfile and Semantic Loss	11
7.1 ProjectionProfile	11
7.2 SemanticLossRecord	11
8 MetatronMicrograph and Lift Report	13
9 Fingerprint Model	15
9.1 Interpretation	15
10 MicroTopologyDiagnostic	17
10.1 Diagnostic Heuristics	17
11 Complement as Void Hypothesis	19

12 Projection Sets and Agreement	20
III Topological Surgery	21
13 TopologicalSurgeryOperator	22
14 Surgery Operator Catalog	23
14.1 AddEdge	23
14.2 RemoveEdge	23
14.3 InsertMediatorRole	23
14.4 SplitOverloadedRole	23
14.5 MergeEquivalentRoles	23
14.6 AddFiber	24
14.7 AddLayerBoundary	24
14.8 IsolateSecurityBoundary	24
14.9 AddConstraint	24
14.10 MarkAntiPattern	24
15 SurgeryOption, Effect and Risk	25
16 Surgery Selection	27
17 CollapsePlan Integration	28
18 Workbench and Foundry Materialization	29
18.1 Workbench Task	29
18.2 Foundry Checks	29
19 Parse-back Topology	31
IV Certification, NormGenes and CorpusCartography	32
20 StructuralCrystal Integration	33
20.1 MetatronCrystalProfile	33
20.2 Constraint Operators	33
21 Metatron Certificate Attachment	35
22 NormGene Integration	36
22.1 Fitness Events	36
23 AntiPattern Integration	38
24 CorpusCartography Integration	39
24.1 MicroTopologyIndex	39
24.2 MicroTopologyRelation	39
25 Metatron Precheck	41
26 Outcome Learning	42
27 Metatron Risk Surface	43

V Gates, API and Implementation	44
28 Gate and Certificate Extensions	45
28.1 Metatron Gates	45
29 Service Traits	46
29.1 MetatronScanKernel	46
29.2 RegionWindowExtractor	46
29.3 TopologicalSurgeryPlanner	46
30 Module Map	48
31 API Contract	49
31.1 HTTP	49
31.2 CLI	49
32 MVP Cut	50
32.1 M1 — Lift and Fingerprint	50
32.2 M2 — Diagnostic and Index	50
32.3 M3 — Gate and Certificate Attachment	50
32.4 M4 — Planning-only Surgery	50
32.5 Deferred	50
33 Implementation Phases	51
34 Acceptance Tests	52
35 System Definition of Done	54
36 Risk Register	55
A PlantUML Diagrams	56
A.1 Metatron Extension Flow	56
A.2 Crystal Attachment	56
A.3 Surgery-backed Collapse	57
B Source Repository Mapping	58
C Final Implementation Maxim	59

Executive Summary

HYPHAE v0.4 already extends the v0.3 CubeSwarm assimilation path into a persistent morphogenic corpus cartography layer. It adds CorpusCartography, StructuralCrystals, AssimilationCertificates, NormGenes, HostTargetCollapsePlans and morphogenic governance. The v0.4.1 extension defined here inserts Metatron-Scan as a microtopological kernel below those v0.4 objects.

```
HYPHAE v0.3:
HostCube -> SourceCubes -> CubeSwarm -> HostTargetDelta
        -> StructuralYield -> GateCascade

HYPHAE v0.4:
StructuralYield -> StructuralCrystalCandidate
                -> AssimilationCertificate
                -> CorpusCartography
                -> NormGene
                -> HostTargetCollapsePlan

HYPHAE v0.4.1:
TopologyRegion -> MetatronMicrograph
                -> MetatronRegionFingerprint
                -> MicroTopologyDiagnostic
                -> TopologicalSurgeryOption
                -> SurgeryBackedCollapseStep
```

The central thesis is:

MetatronScanKernel is the microtopological microscope and scalpel beneath HYPHAE. It fingerprints bounded HDAG/Cube/Motif regions, diagnoses local topology, identifies complement-shaped void hypotheses, proposes structural surgery options and records recurring microstructure in CorpusCartography.

Metatron-Scan must not become a second authority surface. A canonical hash, spectrum, catalog match, complement relation or surgery success may strengthen evidence. None of them may bypass GateCascade, DualFabric certification, Foundry validation, license gates, taint gates, provenance requirements or Kosmocrates governance.

Normative Language and Primary Terms

The words **must**, **must not**, **should**, and **may** are normative. A conformant implementation satisfies all **must** and **must not** statements. A **should** statement may be relaxed only with explicit evidence and an audit record.

Primary Terms

MetatronScanKernel. The bounded microtopology kernel used by HYPHAE to fingerprint, classify, diagnose and propose structural transformations for local topology regions.

TopologyRegionRef. A reference to a bounded region of a CodeHDAG, RepositoryCube, HostVoid, MotifStructuralCore, StructuralCrystal, NormGene or CollapseDimension.

MetatronMicrograph. A small projected graph derived from a HYPHAE topology region under an explicit extraction and projection profile.

SemanticLossRecord. A required record describing which direction, edge kinds, roles, layers or fibers were collapsed during projection into a micrograph.

MetatronRegionFingerprint. A canonical microtopology signature containing canonical hash, orbit size, stabilizer order, spectrum, complement signature, graph traits and evidence.

MicroTopologyDiagnostic. A diagnostic result classifying stability, ambiguity, missing edges, missing roles, weak boundaries, anti-pattern candidates and NormGene proximity.

TopologicalSurgeryOption. A structural planning suggestion composed of topology operations. It is not a code patch.

SurgeryBackedCollapseStep. A HostTargetCollapsePlan step refined by a Metatron diagnostic and a selected TopologicalSurgeryOption.

MicroTopologyIndex. The CorpusCartography index over recurring Metatron fingerprints, catalog classes, anti-patterns, NormGene support and prior surgery outcomes.

Part I

Doctrine and Scope

1 Purpose and Scope

1.1 Purpose

The purpose of this specification is to integrate the existing Metatron-Scan graph analysis kernel into HYPHAE as a bounded microtopological laboratory. The extension enables HYPHAE to recognize local structural forms, measure symmetry and ambiguity, derive complement-based void hypotheses and propose structural surgery options for CollapsePlans.

1.2 Scope

This specification covers:

- neutralization of the existing Metatron-Scan repository into HYPHAE invariants;
- RegionWindowExtractor and ProjectionProfile;
- MetatronMicrograph, SemanticLossRecord and MicrographLiftReport;
- MetatronRegionFingerprint, MicroTopologyDiagnostic and TopologyAmbiguityProfile;
- ComplementVoidHypothesis and TopologicalSurgeryOption;
- SurgeryBackedCollapseStep and WorkbenchPlanning integration;
- MetatronCrystalProfile and MetatronCertificateAttachment;
- NormGeneMicroTopologyProfile and AntiPatternMicroTopologyProfile;
- MicroTopologyIndex and MicroTopologyRelation in CorpusCartography;
- API, CLI, MVP cut, acceptance tests and Definition of Done.

1.3 Non-goals

MetatronScanKernel does not:

- directly edit code;
- directly mutate host repositories;
- replace CodeHDAG, SourceCube, CubeSwarm or CorpusCartography;
- prove semantic equivalence by graph isomorphism alone;
- treat spectral equality as isomorphism;
- promote trusted norms;
- bypass GateCascade, DualFabric certification, Foundry or Kosmocrates governance.

2 Core Doctrine

METATRON-000

MetatronScanKernel analyzes and proposes structural transformations. It must not directly mutate host code, trusted memory, governance, tool authority or Corpus-Cartography.

METATRON-001

MetatronScanKernel must operate on bounded topology regions, not unbounded repositories.

METATRON-002

Every MetatronRegionFingerprint must reference source region, extraction profile, projection profile, semantic-loss record, taint, license and evidence.

METATRON-003

Canonical hash may support recognition but must not alone establish semantic equivalence.

METATRON-004

Spectral equality must not be treated as isomorphism.

METATRON-005

Complement analysis may propose missing structure, but it must pass HostAlignment, GateCascade and Foundry requirements before operational use.

METATRON-006

TopologicalSurgeryOptions are structural planning suggestions, not code patches.

METATRON-007

Metatron-derived diagnostics may influence Motif scoring, Crystal constraints, NormGene fitness and CollapsePlan ordering, but must not bypass gates.

METATRON-008

High-ambiguity regions should require additional HDAG, role, layer, fiber or human-review evidence.

METATRON-009

MetatronScanKernel must preserve inherited taint, license and provenance.

3 Source Repository Neutralization

3.1 Inspected Repository Invariants

The uploaded Metatron-Scan repository already contains a graph-analysis core. The relevant source modules are neutralized as follows.

Repository surface	Existing function	HYPHAE invariant
src/orbit.rs	canonical form, canonical hash, orbit size	CanonicalRegionFingerprint and variant-space signal
src/stabilizer.rs	stabilizer elements, order, classification, generators	Symmetry, rigidity and underdetermination signal
src/spectrum.rs	eigenvalues, spectral radius, algebraic connectivity, triangle count, energy, Laplacian spectrum, spanning trees	Resonance, connectivity and ambiguity signals
src/complement.rs	complement graph, matrix powers, triangle count, trace	ComplementVoidHypothesis and missing-structure candidate extraction
src/catalog.rs	PeriodicTable, catalog entries, catalog lookup	MicroTopologyCatalog and class matching
src/properties.rs	graph property computation	GraphPropertySignature
src/analysis.rs	cospectral analysis, matrix-power analysis, self-complementary analysis, spectral distribution	Ambiguity and pre-filter diagnostics
src/adapters/*	JSON graph, edge list, cargo deps, rust imports, state machine, decompose, ingest report	Prototype input adapters; replaced by HYPHAE RegionWindowExtractor

3.2 Interpretive Rebinding

The existing repo is not imported as a standalone system. It is rebound as:

```
Metatron-Scan original:
small graph analysis and periodic graph catalog
```

HYPHAE rebinding:
bounded microtopology kernel for HDAG/Cube/Motif regions

Core invariants:
canonical form
orbit
stabilizer
spectrum
complement
graph properties
catalog match
cospectral ambiguity

NEUT-METATRON-001

Metatron-Scan may contribute graph invariants, adapters and catalog logic. It must not be imported as an independent authority system.

NEUT-METATRON-002

Existing Metatron input adapters may inform implementation, but HYPHAE integration must use HYPHAE-owned TopologyRegionRef, RegionExtractionProfile and ProjectionProfile.

Part II

Metatron Microtopology Model

4 Architecture Position

MetatronScanKernel sits below HYPHAE's macro-objects.

```
Macro level:
HostCube / SourceCube / CubeSwarm / CompositeSupportCube
StructuralYield / StructuralCrystal / NormGene / CollapsePlan
```

```
Micro level:
TopologyRegionRef
-> RegionWindowExtractor
-> ProjectionProfile
-> MetatronMicrograph
-> MetatronRegionFingerprint
-> MicroTopologyDiagnostic
-> TopologicalSurgeryOption
```

The extension points are:

HYPHAE point	Metatron extension
MotifExtraction	Fingerprint local MotifStructuralCore regions.
StructuralCrystalCandidate	Attach MetatronCrystalProfile and use Metatron constraints.
DualFabricGateCascade	Add lift-quality, semantic-loss and ambiguity gates.
NormGene	Add NormGeneMicroTopologyProfile and micro-topology fitness events.
CorpusCartography	Add MicroTopologyIndex and MicroTopologyRelation.
HostTargetCollapsePlan	Refine CollapseSteps with TopologicalSurgeryOptions.
Foundry	Add parse-back topology validation.

5 TopologyRegionRef

```
pub struct TopologyRegionRef {
  pub id: Digest,
  pub region_kind: TopologyRegionKind,

  pub host_cube_id: Option<Digest>,
  pub source_cube_id: Option<Digest>,
  pub composite_support_cube_id: Option<Digest>,
  pub code_hdag_id: Digest,

  pub node_refs: Vec<HDAGNodeRef>,
  pub edge_refs: Vec<HDAGEdgeRef>,
  pub cube_region_refs: Vec<CubeRegionRef>,

  pub evidence_refs: Vec<EvidenceRef>,
}

pub enum TopologyRegionKind {
  HostVoidRegion,
  MotifCoreRegion,
  StructuralCrystalRegion,
  NormGeneRegion,
  CollapseDimensionRegion,
  AntiPatternRegion,
  BridgeEndpointRegion,
  SourceSupportRegion,
}
```

REGION-001

Every Metatron analysis must start from a bounded TopologyRegionRef.

REGION-002

Every TopologyRegionRef must be evidence-backed and tied to CodeHDAG, CubeRegion or MotifCore provenance.

6 Region Extraction

6.1 RegionExtractionProfile

```
pub struct RegionExtractionProfile {
  pub id: Digest,
  pub name: String,

  pub max_nodes: usize,
  pub max_edges: usize,
  pub max_hops: usize,

  pub anchor_policy: RegionAnchorPolicy,
  pub node_filter: RegionNodeFilter,
  pub edge_filter: RegionEdgeFilter,

  pub include_layers: bool,
  pub include_semantic_fibers: bool,
  pub include_temporal_fibers: bool,
  pub include_host_void_context: bool,

  pub extraction_budget: RegionExtractionBudget,
}

pub enum RegionAnchorPolicy {
  AroundHostVoid,
  AroundMotifCore,
  AroundFailedFoundryNode,
  AroundBoundary,
  AroundTestFiber,
  AroundSecurityBoundary,
  AroundDependencyCluster,
  AroundBridgeEndpoint,
}
```

6.2 MVP Extraction Profiles

Profile	Max nodes	Anchors
TestFiberWindow	8	Route, Handler, Test, Fixture, Assertion
InterfaceBoundary- Window	8	Caller, Interface, Implementation, Con- sumer
LayerBypassWindow	7	Handler, Service, Repository, Database

Profile	Max nodes	Anchors
SecurityBoundary-Window	8	Route, AuthGuard, Capability, Secret, ToolBoundary
StateMachineWindow	8	State, Transition, Guard, Event

7 ProjectionProfile and Semantic Loss

7.1 ProjectionProfile

```
pub struct ProjectionProfile {
  pub id: Digest,
  pub name: String,

  pub graph_kind: ProjectedGraphKind,
  pub node_projection: NodeProjectionPolicy,
  pub edge_projection: EdgeProjectionPolicy,

  pub preserve_roles_as_colors: bool,
  pub preserve_edge_kinds_as_layers: bool,
  pub preserve_direction_as_annotation: bool,

  pub allow_multigraph_collapse: bool,
  pub allow_direction_collapse: bool,
  pub allow_label_collapse: bool,

  pub semantic_loss_policy: SemanticLossPolicy,
}

pub enum ProjectedGraphKind {
  SimpleUndirected,
  SimpleDirected,
  ColoredSimple,
  LayeredSimple,
  BipartiteProjection,
  RoleIncidenceGraph,
}
```

7.2 SemanticLossRecord

```
pub struct SemanticLossRecord {
  pub id: Digest,
  pub projection_profile_id: Digest,

  pub lost_direction_count: usize,
  pub collapsed_edge_kinds: Vec<CodeEdgeKind>,
  pub collapsed_role_kinds: Vec<RoleKind>,
  pub collapsed_layers: Vec<CubeLayerKind>,
  pub collapsed_fibers: Vec<FiberKind>,
}
```

```
pub loss_severity: Q16,  
pub requires_secondary_check: bool,  
pub evidence_refs: Vec<EvidenceRef>,  
}
```

MICROGRAPH-LOSS-001

Metatron fingerprints derived from lossy projections must carry a SemanticLoss-Record.

MICROGRAPH-LOSS-002

High semantic loss must prevent StructuralCrystal certification unless additional HDAG, role, fiber, layer or human-review checks compensate for it.

8 MetatronMicrograph and Lift Report

```
pub struct MetatronMicrograph {
  pub id: Digest,

  pub source_region_ref: TopologyRegionRef,
  pub extraction_profile_id: Digest,
  pub projection_profile_id: Digest,

  pub n: usize,
  pub edges: Vec<(usize, usize)>,

  pub node_backrefs: BTreeMap<usize, HDAGNodeRef>,
  pub edge_backrefs: BTreeMap<(usize, usize), Vec<HDAGEdgeRef>>,

  pub node_role_annotations: BTreeMap<usize, Vec<RoleKind>>,
  pub edge_kind_annotations: BTreeMap<(usize, usize), Vec<CodeEdgeKind>>,

  pub semantic_loss: SemanticLossRecord,
  pub evidence_refs: Vec<EvidenceRef>,
}
```

```
pub struct MicrographLiftReport {
  pub id: Digest,

  pub region_ref: TopologyRegionRef,
  pub micrograph_id: Digest,

  pub extraction_profile_id: Digest,
  pub projection_profile_id: Digest,

  pub node_mapping_complete: bool,
  pub edge_mapping_complete: bool,
  pub semantic_loss: SemanticLossRecord,

  pub lift_quality: Q16,
  pub warnings: Vec<MicrographLiftWarning>,

  pub evidence_refs: Vec<EvidenceRef>,
}
```

MICROGRAPH-001

Every extracted micrograph must record extraction and projection profiles.

MICROGRAPH-002

Every projection must preserve backrefs to original HDAG nodes and edges.

MICROGRAPH-003

Low-quality MicrographLiftReport must block use of Metatron fingerprints for StructuralCrystal certification.

9 Fingerprint Model

```
pub struct MetatronRegionFingerprint {
  pub id: Digest,

  pub source_region: TopologyRegionRef,
  pub extraction_profile_id: Digest,
  pub projection_profile_id: Digest,

  pub node_count: usize,
  pub edge_count: usize,

  pub canonical_hash: String,
  pub canonical_form_digest: Digest,
  pub catalog_id: Option<String>,

  pub orbit_size: usize,
  pub stabilizer_order: usize,
  pub stabilizer_classification: Option<String>,

  pub spectral_signature: SpectralSignature,
  pub complement_signature: ComplementSignature,
  pub property_signature: GraphPropertySignature,

  pub ambiguity: TopologyAmbiguityProfile,
  pub semantic_loss: SemanticLossRecord,

  pub evidence_refs: Vec<EvidenceRef>,
}
```

9.1 Interpretation

Signal	HYPHAE interpretation
canonical hash	Recognition of the same projected local form across runs and repositories.
orbit size	Variant-space size under relabeling.
stabilizer order	Symmetry, rigidity and possible underdetermination.
spectrum	Coupling, resonance and ambiguity signal.
complement	Missing-edge or missing-role hypothesis source.
catalog class	Known microstructure class; not a quality mark.
cospectral ambiguity	Warning that spectrum alone is insufficient.

FINGERPRINT-001

Canonical hash may group projected micrographs. It must not prove semantic equivalence without compatible projection profile, role/fiber/layer evidence and gate checks.

FINGERPRINT-002

Spectrum must be treated as a signal. It must not be treated as isomorphism or semantic equivalence.

10 MicroTopologyDiagnostic

```
pub struct MicroTopologyDiagnostic {
  pub id: Digest,
  pub fingerprint_id: Digest,

  pub topology_class: MicroTopologyClass,
  pub stability: Q16,
  pub rigidity: Q16,
  pub symmetry: Q16,
  pub ambiguity: Q16,

  pub missing_edges: Vec<MissingEdgeHypothesis>,
  pub excess_edges: Vec<ExcessEdgeHypothesis>,
  pub missing_roles: Vec<MissingRoleHypothesis>,
  pub weak_boundaries: Vec<WeakBoundaryHypothesis>,

  pub candidate_antipatterns: Vec<AntiPatternKind>,
  pub candidate_norm_genes: Vec<NormGeneRef>,

  pub evidence_refs: Vec<EvidenceRef>,
}

pub enum MicroTopologyClass {
  Empty,
  Path,
  Star,
  Cycle,
  Clique,
  Bipartite,
  Tree,
  Threshold,
  SelfComplementary,
  StateMachineLike,
  LayerChainLike,
  TestFiberLike,
  InterfaceBoundaryLike,
  SecurityBoundaryLike,
  Unknown,
}
```

10.1 Diagnostic Heuristics

```
high symmetry + low role binding
= underdetermined region, more context needed
```

low symmetry + high host fit
= specific stable structure, possible Crystal candidate

complement close to known NormGene
= surgical repair candidate

cospectral ambiguity
= spectrum alone is insufficient; require more invariants

DIAG-001

Ambiguity is evidence. It must be preserved, not hidden.

DIAG-002

High ambiguity must lower certificate rigor or require additional checks.

11 Complement as Void Hypothesis

```
pub struct ComplementVoidHypothesis {  
    pub id: Digest,  
  
    pub region_fingerprint: MetatronRegionFingerprintRef,  
    pub complement_signature: ComplementSignature,  
  
    pub missing_edge_candidates: Vec<MissingEdgeHypothesis>,  
    pub missing_role_candidates: Vec<MissingRoleHypothesis>,  
    pub missing_fiber_candidates: Vec<MissingFiberHypothesis>,  
  
    pub mapped_host_voids: Vec<TopologicalVoidRef>,  
    pub confidence: Q16,  
  
    pub evidence_refs: Vec<EvidenceRef>,  
}
```

COMPLEMENT-001

Complement analysis may propose HostVoid refinements.

COMPLEMENT-002

Complement-derived missing edges must be interpreted through role, layer, fiber and domain context before Workbench use.

COMPLEMENT-003

Complement analysis must not create HostTargetDelta without HostAlignment and GateCascade.

12 Projection Sets and Agreement

A high-impact region may need multiple projections.

```
pub struct RegionProjectionSet {  
  pub id: Digest,  
  pub region_ref: TopologyRegionRef,  
  pub projections: Vec<MetatronMicrograph>,  
  pub fingerprints: Vec<MetatronRegionFingerprint>,  
  pub aggregate_diagnostic: MicroTopologyDiagnostic,  
}
```

```
pub struct ProjectionAgreementReport {  
  pub id: Digest,  
  pub region_ref: TopologyRegionRef,  
  
  pub projection_fingerprints: Vec<MetatronRegionFingerprintRef>,  
  pub agreement_score: Q16,  
  pub disagreement_reasons: Vec<ProjectionDisagreementReason>,  
  
  pub decision: ProjectionAgreementDecision,  
}
```

PROJECTION-001

High-impact certification should use more than one projection when edge types, roles, direction or fibers materially affect meaning.

PROJECTION-002

Projection disagreement should increase ambiguity and may require HumanReview or additional evidence.

Part III

Topological Surgery

13 TopologicalSurgeryOperator

```
pub struct TopologicalSurgeryOperator {
  pub id: Digest,
  pub name: String,

  pub operator_kind: TopologicalSurgeryKind,
  pub input_requirements: SurgeryInputRequirements,
  pub preconditions: Vec<SurgeryPrecondition>,
  pub operation_template: SurgeryOperationTemplate,

  pub expected_effect: SurgeryEffectModel,
  pub risk_model: SurgeryRiskModel,
  pub validation_template: SurgeryValidationTemplate,

  pub permitted_targets: Vec<SurgeryTargetSurface>,
}

pub enum SurgeryTargetSurface {
  CollapsePlan,
  StructuralYield,
  StructuralCrystalCandidate,
  NormGeneCandidate,
  AntiPatternEvidence,
  WorkbenchPlanningArtifact,
  HumanReviewPacket,
}
```

SURGERY-000

TopologicalSurgeryOperators operate on HYPHAE structural topology, not source code.

SURGERY-001

Every SurgeryOperator must declare input requirements, preconditions, expected effect, risk model and validation template.

14 Surgery Operator Catalog

14.1 AddEdge

```
TopologicalSurgeryKind::AddEdge { edge_kind: CodeEdgeKind }
```

Examples: Test tests Route, Service uses RepositoryPort, Handler calls Service, Fixture configures Test.

14.2 RemoveEdge

```
TopologicalSurgeryKind::RemoveEdge { edge_kind: CodeEdgeKind }
```

Examples: remove Handler to DirectDbQuery, remove UI to Persistence, remove ExternalInput to ToolCall direct edge.

SURGERY-REMOVE-001

RemoveEdge operations affecting runtime behavior must require Foundry validation.

14.3 InsertMediatorRole

```
TopologicalSurgeryKind::InsertMediatorRole { role_kind: RoleKind }
```

Typical mediators: Service, RepositoryPort, AuthGuard, Validator, Adapter, CapabilityBoundary, EventSchema and PluginBoundary.

14.4 SplitOverloadedRole

```
TopologicalSurgeryKind::SplitOverloadedRole {  
  original_role: RoleKind,  
  resulting_roles: Vec<RoleKind>,  
}
```

14.5 MergeEquivalentRoles

```
TopologicalSurgeryKind::MergeEquivalentRoles { role_kind: RoleKind }
```

SURGERY-MERGE-001

MergeEquivalentRoles requires semantic equivalence evidence beyond canonical topology hash.

14.6 AddFiber

```
TopologicalSurgeryKind::AddFiber { fiber_kind: FiberKind }
```

Fiber kinds include TestFiber, InterfaceFiber, SecurityFiber, ErrorRepairFiber, TemporalLifecycleFiber and CapabilityFiber.

14.7 AddLayerBoundary

```
TopologicalSurgeryKind::AddLayerBoundary { boundary_kind: CubeLayerKind }
```

14.8 IsolateSecurityBoundary

```
TopologicalSurgeryKind::IsolateSecurityBoundary {  
  boundary_kind: SecurityBoundaryKind,  
}
```

SURGERY-SECURITY-001

Any surgery that changes security, taint, capability or tool boundaries must require security-specific validation.

14.9 AddConstraint

```
TopologicalSurgeryKind::AddConstraint {  
  constraint_kind: ConstraintOperatorKind,  
}
```

14.10 MarkAntiPattern

```
TopologicalSurgeryKind::MarkAntiPattern {  
  anti_pattern_kind: AntiPatternKind,  
}
```

15 SurgeryOption, Effect and Risk

```
pub struct TopologicalSurgeryOption {
  pub id: Digest,

  pub source_fingerprint: MetatronRegionFingerprint,
  pub target_fingerprint: Option<MetatronRegionFingerprint>,

  pub surgery_kind: TopologicalSurgeryKind,
  pub operation_sequence: Vec<TopologicalSurgeryOp>,

  pub expected_effect: SurgeryEffect,
  pub risk: SurgeryRisk,
  pub validation_requirements: Vec<FoundryCheck>,

  pub evidence_refs: Vec<EvidenceRef>,
}
```

```
pub enum TopologicalSurgeryOp {
  AddEdge(CodeEdgeKind),
  RemoveEdge(CodeEdgeKind),
  InsertRole(RoleKind),
  SplitRole(RoleKind),
  MergeRoles(RoleKind),
  AddFiber(FiberKind),
  AddLayerBoundary(CubeLayerKind),
  AddConstraint(ConstraintOperatorKind),
  MarkAntiPattern(AntiPatternKind),
}
```

```
pub struct SurgeryEffect {
  pub expected_void_reduction: Q16,
  pub expected_mesh_density_gain: Q16,
  pub expected_layer_coherence_gain: Q16,
  pub expected_test_fiber_gain: Q16,
  pub expected_security_boundary_gain: Q16,
  pub expected_oracle_reduction: Q16,
  pub expected_foundry_pass_probability: Q16,
}
```

```
pub struct SurgeryRisk {
  pub aggregate_risk: Q16,
  pub behavior_change_risk: Q16,
  pub regression_risk: Q16,
  pub security_risk: Q16,
  pub taint_risk: Q16,
  pub license_risk: Q16,
  pub semantic_loss_risk: Q16,
```

```
pub ambiguity_risk: Q16,  
pub foundry_failure_risk: Q16,  
}
```

16 Surgery Selection

```
selection_score =  
    expected_void_reduction  
    * expected_foundry_pass_probability  
    * norm_gene_compatibility  
    * evidence_quality  
    * (1 - aggregate_risk)  
    * (1 - ambiguity_risk)  
    * (1 - semantic_loss_risk)
```

SURGERY-SELECT-001

Selection score must not override safety, license, taint, provenance, governance or Foundry gates.

```
pub enum SurgerySelectionDecision {  
    Select,  
    SelectWithHumanReview,  
    KeepAsEvidenceOnly,  
    RequireMoreContext,  
    RejectAll,  
}
```

17 CollapsePlan Integration

```
pub struct SurgeryBackedCollapseStep {  
  pub id: Digest,  
  
  pub collapse_step_id: Digest,  
  pub collapse_dimension_refs: Vec<CollapseDimensionRef>,  
  
  pub source_region: TopologyRegionRef,  
  pub source_fingerprint: MetatronRegionFingerprintRef,  
  pub diagnostic: MicroTopologyDiagnosticRef,  
  
  pub selected_surgery: TopologicalSurgeryOptionRef,  
  pub alternative_surgeries: Vec<TopologicalSurgeryOptionRef>,  
  
  pub validation_plan: CollapseValidationPlan,  
  pub evidence_refs: Vec<EvidenceRef>,  
}
```

Mapping:

Surgery kind	Collapse mapping
InsertMediatorBoundary	Singularity or synchronized group
AddTestRelation	Leaf step
SplitOverloadedRole	Singularity
AddSecurityBoundary	Group plus security review
MarkAntiPattern	AntiPatternEvidence and GateGuard
RemoveEdge	High-risk explicit step with Foundry validation

SURGERY-COLLAPSE-001

SurgeryBackedCollapseSteps must be explicit in HostTargetCollapsePlan.

SURGERY-COLLAPSE-002

High-risk surgery must require HumanReview, Foundry validation or both.

18 Workbench and Foundry Materialization

18.1 Workbench Task

```
pub struct SurgeryWorkbenchTask {
  pub id: Digest,

  pub surgery_backed_step_id: Digest,
  pub task_kind: SurgeryWorkbenchTaskKind,

  pub structural_intent: String,
  pub required_changes: Vec<StructuralChangeIntent>,
  pub forbidden_changes: Vec<ForbiddenChangeIntent>,

  pub validation_plan: CollapseValidationPlan,
  pub evidence_refs: Vec<EvidenceRef>,
}

pub enum SurgeryWorkbenchTaskKind {
  IntroduceBoundary,
  AddTestStructure,
  RefactorLayerDependency,
  IsolateSecurityFlow,
  AddConstraintCheck,
  RemoveBypass,
  SplitResponsibility,
  CreateAdapter,
  HumanReviewDesignDecision,
}
```

18.2 Foundry Checks

```
pub enum SurgeryFoundryCheck {
  Build,
  UnitTests,
  IntegrationTests,
  TypeCheck,
  Lint,
  SecurityScan,
  ParseBackTopology,
  TopologyDiff,
  RegressionSnapshot,
  ContractTest,
```

```
TaintFlowCheck,  
}
```

SURGERY-FOUNDRY-001

Any surgery that may affect executable behavior must require Foundry validation.

SURGERY-FOUNDRY-002

Surgery validation should include parse-back topology comparison when feasible.

19 Parse-back Topology

```
pub struct TopologyExpectation {  
    pub id: Digest,  
    pub expected_added_edges: Vec<CodeEdgeKind>,  
    pub expected_removed_edges: Vec<CodeEdgeKind>,  
    pub expected_added_roles: Vec<RoleKind>,  
    pub expected_removed_antipatterns: Vec<AntiPatternKind>,  
    pub expected_new_fibers: Vec<FiberKind>,  
    pub expected_boundary_state: Option<BoundaryState>,  
}
```

```
pub struct SurgeryParseBackReport {  
    pub id: Digest,  
    pub before_fingerprint: MetatronRegionFingerprintRef,  
    pub expected_target: MetatronTargetTopology,  
    pub after_fingerprint: MetatronRegionFingerprintRef,  
  
    pub topology_expectations_met: bool,  
    pub unexpected_edges: Vec<HDAGEdgeRef>,  
    pub missing_expected_edges: Vec<CodeEdgeKind>,  
    pub residual_antipatterns: Vec<AntiPatternKind>,  
  
    pub evidence_refs: Vec<EvidenceRef>,  
}
```

SURGERY-PARSEBACK-001

A passing build with failed topology expectation should not count as full surgery success.

Part IV

Certification, NormGenes and CorpusCartography

20 StructuralCrystal Integration

20.1 MetatronCrystalProfile

```
pub struct MetatronCrystalProfile {
  pub id: Digest,

  pub projection_set_id: Digest,
  pub primary_fingerprint: MetatronRegionFingerprintRef,
  pub secondary_fingerprints: Vec<MetatronRegionFingerprintRef>,

  pub required_catalog_classes: Vec<String>,
  pub forbidden_catalog_classes: Vec<String>,

  pub complement_requirements: Vec<ComplementRequirement>,
  pub ambiguity_ceiling: Q16,
  pub semantic_loss_ceiling: Q16,

  pub diagnostic_refs: Vec<MicroTopologyDiagnosticRef>,
  pub lift_reports: Vec<MicrographLiftReportRef>,

  pub evidence_refs: Vec<EvidenceRef>,
}
```

20.2 Constraint Operators

```
pub enum ConstraintOperatorKind {
  RoleInvariant,
  EdgeInvariant,
  FiberInvariant,
  LayerInvariant,
  PhaseInvariant,

  MetatronCanonicalFingerprint,
  MetatronCatalogClass,
  MetatronComplementRequirement,
  MetatronAmbiguityBound,
  MetatronSemanticLossBound,
  MetatronProjectionAgreement,

  HostVoidCoverageInvariant,
  ConflictBound,
  LicenseTaintBound,
  FoundryOutcomeBound,
}
```

METATRON-CRYSTAL-001

A StructuralCrystalCandidate may include MetatronRegionFingerprints as structural invariants.

METATRON-CRYSTAL-002

Metatron fingerprint equality alone must not prove semantic equivalence.

METATRON-CRYSTAL-003

Cospectral or high-ambiguity regions must require additional HDAG, role, fiber, layer or evidence checks before certification.

21 Metatron Certificate Attachment

```
pub struct MetatronCertificateAttachment {
  pub id: Digest,

  pub structural_crystal_candidate_id: Digest,

  pub projection_set_id: Digest,
  pub fingerprints: Vec<MetatronRegionFingerprintRef>,
  pub diagnostics: Vec<MicroTopologyDiagnosticRef>,
  pub surgery_options: Vec<TopologicalSurgeryOptionRef>,

  pub semantic_loss_summary: SemanticLossSummary,
  pub ambiguity_summary: TopologyAmbiguitySummary,
  pub projection_agreement_report: Option<ProjectionAgreementReportRef>,

  pub metatron_gate_results: Vec<MetatronGateResult>,

  pub decision: MetatronAttachmentDecision,
  pub evidence_refs: Vec<EvidenceRef>,
}

pub enum MetatronAttachmentDecision {
  SupportsCertification,
  SupportsEvidenceOnly,
  AmbiguousRequiresReview,
  ConflictedReject,
  NotApplicable,
}
```

METATRON-CERT-001

MetatronCertificateAttachment may strengthen a certificate only if lift quality, semantic loss, ambiguity and projection agreement gates pass.

METATRON-CERT-002

Metatron attachment may downgrade certification to EvidenceOnly or HumanReview if local topology is ambiguous, lossy or conflicted.

22 NormGene Integration

```
pub struct NormGeneMicroTopologyProfile {
  pub id: Digest,

  pub required_fingerprints: Vec<MetatronRegionFingerprintRef>,
  pub accepted_catalog_classes: Vec<String>,
  pub forbidden_catalog_classes: Vec<String>,

  pub required_topology_traits: Vec<GraphPropertyRequirement>,
  pub forbidden_topology_traits: Vec<GraphPropertyRequirement>,

  pub complement_requirements: Vec<ComplementRequirement>,
  pub ambiguity_ceiling: Q16,
  pub semantic_loss_ceiling: Q16,

  pub projection_profiles: Vec<ProjectionProfileRef>,
  pub evidence_refs: Vec<EvidenceRef>,
}
```

22.1 Fitness Events

```
pub enum NormFitnessEventKind {
  SupportedBySourceCube,
  SupportedByStructuralCrystal,
  UsedAsContextHint,
  FoundryPassed,
  FoundryFailed,

  MetatronFingerprintMatched,
  MetatronProjectionAgreed,
  MetatronSurgerySucceeded,
  MetatronSurgeryFailed,
  MetatronAmbiguityIncreased,
  MetatronAntiPatternDetected,

  ConflictDetected,
  SupersededByBetterGene,
  HumanApproved,
  HumanRejected,
}
```

METATRON-FITNESS-001

Metatron fingerprint recurrence may increase NormGene support only when projection profiles, semantic-loss bounds and role/fiber evidence are compatible.

METATRON-FITNESS-002

Metatron surgery failure should reduce NormGene fitness if the failure is attributable to the NormGene's microtopology assumption.

METATRON-NORM-BIRTH-001

Repeated Metatron fingerprint recurrence alone must not birth a NormGene.

23 AntiPattern Integration

```
pub struct AntiPatternMicroTopologyProfile {  
  pub id: Digest,  
  
  pub forbidden_fingerprints: Vec<MetatronRegionFingerprintRef>,  
  pub forbidden_catalog_classes: Vec<String>,  
  
  pub diagnostic_shape: MicroTopologyDiagnosticRef,  
  pub avoidance_surgeries: Vec<TopologicalSurgeryOptionRef>,  
  
  pub risk_score: Q16,  
  pub evidence_refs: Vec<EvidenceRef>,  
}
```

Examples:

```
HandlerDirectDbAccess  
ToolCallWithoutTaintBoundary  
PluginWithoutCapabilityBoundary  
TestWithoutAssertion  
AuthCheckAfterSideEffect  
SharedMutableStateBridge
```

24 CorpusCartography Integration

24.1 MicroTopologyIndex

```
pub struct MicroTopologyIndex {
  pub entries: BTreeMap<Digest, MetatronRegionFingerprint>,

  pub by_canonical_hash: BTreeMap<String, Vec<Digest>>,
  pub by_catalog_id: BTreeMap<String, Vec<Digest>>,
  pub by_region_kind: BTreeMap<TopologyRegionKind, Vec<Digest>>,
  pub by_motif_kind: BTreeMap<MotifKind, Vec<Digest>>,
  pub by_norm_gene: BTreeMap<NormGeneRef, Vec<Digest>>,
  pub by_antipattern: BTreeMap<AntiPatternKind, Vec<Digest>>,
  pub by_surgery_kind: BTreeMap<TopologicalSurgeryKind, Vec<Digest>>,
}
```

```
pub struct CorpusCartography {
  pub id: Digest,
  pub scope: CorpusScope,

  pub source_index: SourceCubeIndex,
  pub relation_index: CorpusRelationIndex,
  pub motif_index: MotifIndex,
  pub norm_gene_index: NormGeneIndex,
  pub structural_crystal_archive: StructuralCrystalArchive,
  pub negative_evidence_index: NegativeEvidenceIndex,
  pub conflict_index: ConflictIndex,

  pub micro_topology_index: Option<MicroTopologyIndex>,

  pub replay_root: Digest,
}
```

24.2 MicroTopologyRelation

```
pub struct MicroTopologyRelation {
  pub id: Digest,

  pub from_fingerprint: MetatronRegionFingerprintRef,
  pub to_fingerprint: MetatronRegionFingerprintRef,

  pub relation_kind: MicroTopologyRelationKind,
  pub strength: Q16,
```

```
pub confidence: Q16,  
  
pub supporting_runs: Vec<Digest>,  
pub supporting_crystals: Vec<StructuralCrystalRef>,  
pub supporting_norm_genes: Vec<NormGeneRef>,  
pub negative_evidence_refs: Vec<NegativeEvidenceRef>,  
  
pub evidence_refs: Vec<EvidenceRef>,  
}  
  
pub enum MicroTopologyRelationKind {  
    SameCanonicalForm,  
    SameCatalogClass,  
    ComplementOf,  
    SurgeryTransformsInto,  
    ForbiddenByAntiPattern,  
    SupportsNormGene,  
    ConflictsWithNormGene,  
    SupersedesLocalShape,  
    AmbiguousWith,  
}
```

CARTO-MICRO-001

Metatron-derived fingerprints may be stored in CorpusCartography only with region refs, projection profiles, semantic-loss records, evidence refs, taint and license state.

CARTO-MICRO-002

MicroTopologyIndex may influence CartographyPrecheck, Frontier ranking, Motif scoring, CollapsePlan ordering and NormGene fitness. It must not bypass gates.

25 Metatron Precheck

```
pub struct MetatronPrecheckHit {
  pub id: Digest,

  pub host_region: TopologyRegionRef,
  pub fingerprint: MetatronRegionFingerprintRef,

  pub matching_crystals: Vec<StructuralCrystalRef>,
  pub matching_norm_genes: Vec<NormGeneRef>,
  pub matching_antipatterns: Vec<AntiPatternGeneRef>,
  pub prior_surgeries: Vec<TopologicalSurgeryOptionRef>,
  pub prior_outcomes: Vec<SurgeryOutcomeRecordRef>,

  pub recommendation: MetatronPrecheckRecommendation,
}

pub enum MetatronPrecheckRecommendation {
  ReuseKnownNormGene,
  AvoidKnownAntiPattern,
  ProposeKnownSurgery,
  RequireMoreContext,
  HumanReview,
  EvidenceOnly,
}
```

26 Outcome Learning

```
pub struct SurgeryOutcomeRecord {
  pub id: Digest,

  pub surgery_option_id: Digest,
  pub workbench_task_id: Digest,
  pub foundry_outcome_id: Digest,
  pub parse_back_report_id: Option<Digest>,

  pub outcome_kind: SurgeryOutcomeKind,
  pub effect_realized: SurgeryEffect,
  pub failure_reason: Option<SurgeryFailureReason>,

  pub evidence_refs: Vec<EvidenceRef>,
}

pub enum SurgeryOutcomeKind {
  Success,
  PartialSuccess,
  BuildPassedTopologyFailed,
  BuildFailed,
  FoundryFailed,
  HumanRejected,
  Unsafe,
  NotApplicable,
}
```

```
pub struct MicroTopologyOutcomeUpdate {
  pub id: Digest,

  pub before_fingerprint: MetatronRegionFingerprintRef,
  pub after_fingerprint: Option<MetatronRegionFingerprintRef>,
  pub surgery_outcome: SurgeryOutcomeRecordRef,

  pub relation_updates: Vec<MicroTopologyRelationUpdate>,
  pub norm_fitness_events: Vec<NormFitnessEvent>,
  pub negative_evidence: Vec<NegativeEvidenceEntry>,

  pub evidence_refs: Vec<EvidenceRef>,
}
```

OUTCOME-MICRO-001

Successful surgery may strengthen MicroTopologyRelation and NormGene fitness.

OUTCOME-MICRO-002

Failed surgery should create NegativeEvidence or risk penalty.

27 Metatron Risk Surface

```
pub struct MetatronRiskSurface {  
  pub id: Digest,  
  
  pub semantic_loss_risk: Q16,  
  pub projection_bias_risk: Q16,  
  pub context_truncation_risk: Q16,  
  pub cospectral_ambiguity_risk: Q16,  
  pub catalog_overfit_risk: Q16,  
  pub complement_overinterpretation_risk: Q16,  
  
  pub aggregate_risk: Q16,  
}
```

RISK-METATRON-001

High MetatronRiskSurface aggregate risk must downgrade use from certification to EvidenceOnly or HumanReview unless compensated by independent HYPHAE evidence.

Part V

Gates, API and Implementation

28 Gate and Certificate Extensions

28.1 Metatron Gates

Gate	Purpose
G19	Micrograph Lift Quality Gate
G20	Semantic Loss Gate
G21	Projection Agreement Gate
G22	MicroTopology Ambiguity Gate
G23	Complement Interpretation Gate
G24	Surgery Outcome Gate
G25	Metatron Risk Surface Gate

METATRON-GATE-001

Metatron gates must be advisory-hardening gates, not authority gates.

METATRON-GATE-002

Failure of Metatron gates may block Metatron-derived evidence from certificate use, but should not invalidate non-Metatron HYPHAE evidence unless directly contradictory.

METATRON-GATE-003

Metatron-derived complement or surgery suggestions must pass HostAlignment, GateCascade and Foundry requirements before Workbench operational use.

29 Service Traits

29.1 MetatronScanKernel

```
pub trait MetatronScanKernel {
    fn fingerprint_micrograph(
        &self,
        graph: &MetatronMicrograph,
    ) -> Result<MetatronRegionFingerprint, MetatronError>;

    fn diagnose_fingerprint(
        &self,
        fingerprint: &MetatronRegionFingerprint,
        context: &MetatronDiagnosticContext,
    ) -> Result<MicroTopologyDiagnostic, MetatronError>;

    fn match_catalog(
        &self,
        fingerprint: &MetatronRegionFingerprint,
    ) -> Result<Option<MicroTopologyCatalogEntry>, MetatronError>;
}
```

29.2 RegionWindowExtractor

```
pub trait RegionWindowExtractor {
    fn extract_region(
        &self,
        input: RegionExtractionInput,
    ) -> Result<TopologyRegionRef, RegionExtractionError>;

    fn project_region(
        &self,
        region: &TopologyRegionRef,
        profile: &ProjectionProfile,
    ) -> Result<MetatronMicrograph, RegionExtractionError>;
}
```

29.3 TopologicalSurgeryPlanner

```
pub trait TopologicalSurgeryPlanner {
    fn propose_surgery(
        &self,
        diagnostic: &MicroTopologyDiagnostic,
```

```
        context: &SurgeryPlanningContext,
    ) -> Result<Vec<TopologicalSurgeryOption>, SurgeryError>;

    fn attach_to_collapse_step(
        &self,
        option: &TopologicalSurgeryOption,
        collapse_context: &CollapsePlanningContext,
    ) -> Result<SurgeryBackedCollapseStep, SurgeryError>;
}
```

30 Module Map

```
kosmo-hyphae/  
  src/  
    metatron/  
      mod.rs  
      kernel.rs  
      region_extraction.rs  
      projection.rs  
      micrograph.rs  
      fingerprint.rs  
      diagnostic.rs  
      surgery.rs  
      gates.rs  
      certificate.rs  
      corpus_index.rs  
      outcome.rs  
      errors.rs
```

Possible later split:

```
crates/  
  kosmo-metatron-scan/  
    src/  
      lib.rs  
      orbit.rs  
      stabilizer.rs  
      spectrum.rs  
      complement.rs  
      catalog.rs  
      analysis.rs  
  
  kosmo-hyphae/  
    src/metatron_bridge/  
      region_extraction.rs  
      projection.rs  
      diagnostics.rs  
      surgery.rs  
      corpus_index.rs
```

Recommendation: keep integration internal for v0.4.1; split the scan core into a crate after API stabilization.

31 API Contract

31.1 HTTP

```
GET /hyphae/runs/{run_id}/metatron/regions
GET /hyphae/runs/{run_id}/metatron/fingerprints
GET /hyphae/runs/{run_id}/metatron/diagnostics
GET /hyphae/runs/{run_id}/metatron/surgeries

GET /hyphae/cartography/{scope}/microtopology
GET /hyphae/crystals/{id}/metatron
GET /hyphae/norm-genes/{id}/microtopology
```

MVP only:

```
GET /hyphae/runs/{run_id}/metatron/fingerprints
GET /hyphae/runs/{run_id}/metatron/diagnostics
GET /hyphae/cartography/{scope}/microtopology
```

31.2 CLI

```
kosmo hyphae metatron regions <run-id>
kosmo hyphae metatron fingerprints <run-id>
kosmo hyphae metatron diagnose <region-id>
kosmo hyphae metatron surgery <region-id>
kosmo hyphae cartography microtopology --scope local
```

32 MVP Cut

32.1 M1 — Lift and Fingerprint

- RegionWindowExtractor
- ProjectionProfile
- MetatronMicrograph
- SemanticLossRecord
- MicrographLiftReport
- MetatronRegionFingerprint

32.2 M2 — Diagnostic and Index

- MicroTopologyDiagnostic
- TopologyAmbiguityProfile
- ComplementVoidHypothesis
- MicroTopologyIndex in CorpusCartography

32.3 M3 — Gate and Certificate Attachment

- LiftQualityGate
- SemanticLossGate
- AmbiguityGate
- MetatronCertificateAttachment

32.4 M4 — Planning-only Surgery

- TopologicalSurgeryOption
- SurgeryBackedCollapseStep
- SurgeryWorkbenchTask
- Foundry validation requirements

32.5 Deferred

- automatic Crystal supersession;
- full MicroTopologyOutcome learning;
- multi-projection optimizer;
- automated Surgery selection in high-risk regions;
- standalone kosmo-metatron-scan crate.

33 Implementation Phases

Phase	Output
M0	Extract Metatron core as a stable library interface.
M1	TopologyRegionRef to MetatronMicrograph with backrefs and semantic-loss record.
M2	Micrograph to existing orbit, stabilizer, spectrum, complement and catalog functions.
M3	Loss, projection warnings and ambiguity profile.
M4	MicroTopologyIndex in CorpusCartography.
M5	MetatronCrystalProfile and MetatronCertificateAttachment.
M6	Planning-only SurgeryOptions and SurgeryBackedCollapseSteps.
M7	Parse-back topology and outcome learning.

34 Acceptance Tests

MTA-001:
A HostVoidRegion can be extracted into a bounded MetatronMicrograph.

MTA-002:
Every MetatronMicrograph preserves HDAG node and edge backrefs.

MTA-003:
Lossy projection emits SemanticLossRecord.

MTA-004:
Fixed region + projection profile produces deterministic canonical_hash.

MTA-005:
Spectrum equality alone is not accepted as semantic equivalence.

MTA-006:
High ambiguity downgrades Metatron evidence to EvidenceOnly or HumanReview.

MTA-007:
Metatron fingerprint can be attached to StructuralCrystalCandidate.

MTA-008:
Metatron fingerprint alone cannot create StructuralCrystalRecord.

MTA-009:
MicroTopologyIndex groups recurring canonical_hash across runs.

MTA-010:
CartographyPrecheck may retrieve prior microtopology but cannot bypass gates.

MTA-011:
Complement-derived MissingEdgeHypothesis does not become HostTargetDelta without HostAlignment.

MTA-012:
TopologicalSurgeryOption creates validation requirements.

MTA-013:
SurgeryBackedCollapseStep is planning guidance, not patch authority.

MTA-014:
Parse-back mismatch creates SurgeryOutcome failure or partial success.

MTA-015:

Successful surgery strengthens MicroTopologyRelation and may update NormGene fitness.

35 System Definition of Done

DOD-MTA-001:

MetatronScanKernel can fingerprint bounded HYPHAE regions.

DOD-MTA-002:

Every fingerprint is traceable to TopologyRegionRef, ProjectionProfile, SemanticLossRecord, evidence, taint and license state.

DOD-MTA-003:

Metatron diagnostics can produce ambiguity and complement hypotheses.

DOD-MTA-004:

MicroTopologyIndex persists recurring fingerprints in CorpusCartography.

DOD-MTA-005:

Metatron attachments can strengthen or downgrade StructuralCrystal certification,
but cannot independently certify.

DOD-MTA-006:

SurgeryOptions are structural planning suggestions only.

DOD-MTA-007:

Surgery-backed CollapseSteps declare Foundry and parse-back validation.

DOD-MTA-008:

Metatron-derived evidence cannot bypass GateCascade, DualFabric certification,
Foundry or Kosmocrates governance.

36 Risk Register

Risk	Mitigation
Graph form confused with semantics	SemanticLossRecord plus role, fiber and layer checks.
Spectrum overtrusted	Spectrum is a signal, not isomorphism.
Small windows lose context	ProjectionAgreement and multi-projection.
Complement treated as code suggestion	ComplementVoidHypothesis must pass HostAlignment and gates.
Surgery treated as patch	SurgeryWorkbenchTask remains planning artifact only.
Catalog overfit	Catalog match is recognition, not quality.
Metatron becomes authority	Metatron gates are hardening/evidence gates only.
High symmetry or ambiguity	HumanReview or larger region extraction.

A PlantUML Diagrams

A.1 Metatron Extension Flow

```
@startuml
start
:Select TopologyRegionRef;
:Extract bounded region;
:Project through ProjectionProfile;
:Emit SemanticLossRecord;
:Build MetatronMicrograph;
:Compute canonical hash / orbit / stabilizer / spectrum / complement;
:Emit MetatronRegionFingerprint;
:Diagnose microtopology;
if (surgery useful?) then (yes)
    :Emit TopologicalSurgeryOption;
    :Attach SurgeryBackedCollapseStep;
else (no)
    :Store diagnostic evidence;
endif
:Update MicroTopologyIndex if durable;
stop
@enduml
```

A.2 Crystal Attachment

```
@startuml
start
:StructuralCrystalCandidate;
:Extract MotifCore region;
:Fingerprint region;
:Run Metatron gates;
if (lift/loss/ambiguity pass?) then (yes)
    :Attach MetatronCrystalProfile;
    :Certificate strengthened;
else (no)
    :Downgrade Metatron evidence;
    :EvidenceOnly or HumanReview;
endif
stop
@enduml
```

A.3 Surgery-backed Collapse

```
@startuml
start
:HostVoid region;
:Metatron diagnostic;
:Propose surgery options;
:Select option under policy;
:Create SurgeryBackedCollapseStep;
:Create WorkbenchPlanningArtifact;
:Foundry validates;
:Parse-back topology;
if (topology expectation met?) then (yes)
    :Surgery success;
    :Strengthen MicroTopologyRelation;
else (no)
    :NegativeEvidence;
    :Increase risk / ambiguity;
endif
stop
@enduml
```

B Source Repository Mapping

Metatron file	Reusable invariant	HYPHAE integration
<code>orbit.rs</code>	canonical form/hash, orbit size	<code>MetatronRegionFingerprint</code>
<code>stabilizer.rs</code>	symmetry group order/classification	rigidity, ambiguity, underdetermination
<code>spectrum.rs</code>	eigenvalues, connectivity, triangles, energy, spanning trees	<code>SpectralSignature</code> and risk signals
<code>complement.rs</code>	complement graph and matrix traces	<code>ComplementVoidHypothesis</code>
<code>catalog.rs</code>	<code>PeriodicTable</code> entries and lookup	<code>MicroTopologyCatalog</code>
<code>analysis.rs</code>	cospectral and matrix-power analysis	ambiguity and prefilter diagnostics
<code>properties.rs</code> <code>adapters/*</code>	graph properties source input prototypes	<code>GraphPropertySignature</code> replaced by <code>RegionWindowExtractor</code>

C Final Implementation Maxim

MetatronScanKernel may make HYPHAE sharper. It must never make HYPHAE less governed.

A fingerprint is recognition. A diagnostic is evidence. A surgery is planning. Only Workbench and Foundry may materialize, and only Kosmocrates governance may promote.